AlgoNexis: Implementation and Performance Analysis of an Interactive Algorithms Portfolio

Shreyash

CSU Global

CSC506: Design and Analysis of Algorithms

[Instructor name]

[Assignment due date]

AlgoNexis: Implementation and Performance Analysis of an Interactive Algorithms Portfolio

AlgoNexis connects algorithm implementation, abstract data types, interactive explanation, and empirical measurement in a single Python portfolio. Its educational purpose is to make a result inspectable: a learner can order a list, find one ranked element, modify a structure, and examine evidence about the work performed. Correct output is necessary, but a useful portfolio also explains the relationship between an interface contract, its internal representation, and its cost. The application therefore exposes operations through four Streamlit sections while keeping the underlying implementation importable without starting a user interface.

The implemented scope comprises Bubble Sort, randomized Quickselect, immutable sorting-event data, a Custom Set, Stack, Queue, Binary Search Tree, undirected Graph, and separate-chaining Hash Table. A manager coordinates explicit operation routes. The presentation and user guide show the actual application; the figures in this report are generated from saved observations. These layers support different questions. Unit tests establish behavior for selected scenarios, interface checks establish integration behavior, and microbenchmarks describe the measured workloads on the recorded computer. None alone proves universal correctness or performance.

Two distinctions guide the analysis. First, sorting returns a complete ordering, whereas selection returns one order statistic. Their elapsed times cannot identify an interchangeable winner. Second, asymptotic complexity describes growth under assumptions; a short run on one computer supplies finite observations. Small measurements can be dominated by validation,

copying, method dispatch, allocation, and scheduling. The report interprets those effects alongside the implementation rather than treating a plotted line as a proof of a complexity class.

The final default run contains 1,080 measured rows and 8 explicit skipped cases. A separate controlled hash experiment contributes 60 measured rows. Five timing repeats and five separate allocation repeats are recorded for each eligible case. Release QA covers algorithm edge cases, structure behavior, visualization progression, navigation, persistence, resets, invalid input, and evidence integrity. The full verification record identifies the executed commands and test results. This evidence establishes a reproducible educational portfolio within the documented bounds, rather than a claim about production throughput or every possible input.

Algorithms and the Cost of Explanation

Bubble Sort copies the caller's integer list, compares adjacent values, and swaps an out-of-order pair. Each pass shortens the unsorted suffix; a pass without a swap ends execution early. Adjacent comparison and shrinking passes follow the conventional structure described by OpenDSA Project Contributors (n.d.). The early-exit behavior is an explicit addition in this implementation. For an already sorted list of n elements, $n-1$ comparisons suffice. Reverse order can require $n(n-1)/2$ comparisons and swaps. The untraced algorithm therefore has $O(n)$ best-case and $O(n^2)$ average/worst-case time, with $O(n)$ auxiliary storage because the public contract copies its input.

Tracing changes the resource model. Initial, compare, swap, and final events contain immutable tuple snapshots, cumulative counters, and highlighted indices. The visualizer reads these records by a zero-based index; it does not rerun the algorithm. There can be $O(n^2)$ events, each containing $O(n)$ values. Consequently, complete tracing can require $O(n^3)$ time and storage. That cost is educational instrumentation, not an intrinsic requirement of adjacent swapping. The

interface limits traced datasets to 50 elements, and the benchmark explicitly disables recording. This separation prevents visualization overhead from being mislabeled as sorting time.

Quickselect also copies its input. Its public rank k is one-based, with repeated values occupying repeated ranks. It rejects an empty list or an out-of-range rank. Iterative three-way partitioning divides a current region into values below, equal to, and above a randomly chosen pivot. Only the region containing the desired rank remains active. Equal values can terminate a selection immediately when the requested rank falls in the equal region. Partition-based rank selection is also represented in Sedgewick and Wayne's (n.d.-c) Quick implementation; the project's three-way partition and explicit nonmutation contract are its own documented choices.

The expected time for independent randomized pivots is O(n): partitioning costs a scan, and useful splits reduce the remaining region geometrically in expectation. Repeated poor splits permit $O(n^2)$ worst-case work. Iteration avoids recursive call-stack growth, but the protective copy still costs O(n) space. An optional Random instance makes pivot sequences reproducible without changing global random state. Tests compare all valid ranks against sorted(values)[k−1], verify duplicates and negatives, and check caller preservation. Thus the API makes both rank meaning and mutation behavior explicit.

Structures, Representations, and Contracts

The Custom Set uses dictionary keys for membership and insertion-ordered display. Union, intersection, difference, and symmetric difference are explicit loops that return new Custom Set instances. The implementation does not delegate set algebra to Python's built-in set operators. Expected constant-time dictionary membership supports linear scans over participating elements, subject to ordinary hashing assumptions. Display order is a usability property; mathematical equality depends on membership. This distinction makes examples

predictable without confusing a set with a sorted sequence. Values must remain hashable, and display copies are shallow rather than recursive copies of arbitrary objects.

The Stack uses the end of a list for last-in, first-out operations. The Queue uses deque for first-in, first-out removal. Python Software Foundation (n.d.-a) documents approximately constant-time operations at deque ends and the element-moving cost of list removal at the front. That representation choice avoids paying O(n) movement for each dequeue. Empty pop, peek, dequeue, and front operations raise IndexError. The benchmark drains n preloaded elements, so an O(n) batch is compatible with O(1) individual removal. Its output list is deliberately included in the measured operation.

The Binary Search Tree stores unique integers and remains unbalanced. Insert and search follow a root-to-node path of height h; traversals visit each stored node. Search is O(h), becoming O(n) for a chain. The comparable unbalanced structure in Sedgewick and Wayne (n.d.-b) likewise documents linear worst-case operations. Sorted insertion therefore has a structural consequence that randomized input may avoid. Iterative traversal prevents recursion depth failures but does not rebalance the tree. Duplicate inputs leave fewer stored nodes than queries, which matters when interpreting the duplicate-pattern benchmark.

The Graph maintains insertion-ordered adjacency dictionaries, adds undirected edges, and supports breadth-first and depth-first traversal of the reachable component. Both inspect vertices and incident edges, giving O(V+E) traversal time with O(V) auxiliary traversal state. Sedgewick and Wayne (n.d.-a) document this standard breadth-first bound. The fixed-capacity Hash Table uses custom bucket lists and separate chaining; it distinguishes absent keys from stored None values. Expected lookup is O(1+n/c) for c buckets under suitable distribution, with O(n) worst-case scans. Sedgewick and Wayne (n.d.-d) explicitly condition constant expected

hashing cost on uniform distribution. A registry of allowed bound methods integrates all six structures without evaluating user expressions or dynamically dispatching arbitrary names.

Measurement Method and Reproducibility

Measurements were collected on AMD Ryzen AI 9 HX 370 w/ Radeon 890M, with 63.12 GiB reported physical memory, Windows-11-10.0.26300-SP0, and CPython 3.14.4. Installed application packages were Streamlit 1.66.1, pandas 3.0.6, Altair 6.3.0, and pytest 9.2.0. The environment sidecar records the exact measured revision, core-file SHA-256 hashes, run identifier, and configuration. The default run identifier is c6f11506577d47dc87eddb2fc4391644. CSV content hashes and all figure hashes appear in results/analysis_manifest.json. Provenance is tied to the measured source revision; later document commits do not imply that a different algorithm was measured.

The default sizes are 10, 100, 1,000, and 5,000. Seed 506 generates integer values in [−10,000, 10,000]; sorted and reverse patterns reorder the same generated sample. The duplicate pattern draws from [−3, 3], so repeated queries can greatly exceed unique stored keys. Each eligible case receives one warm-up, then five paired repetitions using fresh fixtures separately for timing and allocation. Quickselect selects k=(n+1)//2 and uses pivot seed 506+trial for both measurement types. All results are checked against independently prepared expectations outside the measured region.

Elapsed time is the difference of perf_counter_ns readings, converted to milliseconds. Python Software Foundation (n.d.-b) describes this counter as suitable for short durations and specifies that the integer variant avoids floating-point timestamp precision loss. Fixture generation, tree construction, expected-result calculation, correctness checks, and export are excluded. Copying inside algorithms and constructing output lists inside operations remain

included. A lock serializes measurements within the process. The standalone command line run also avoids concurrent interface rendering, although operating-system scheduling and background activity remain possible sources of variation.

Allocation trials start tracemalloc immediately before the operation, read its peak, and stop it in a finally block. These are incremental Python-tracked allocations, not resident RAM, total structure storage, or all native-library allocation (Python Software Foundation, n.d.-c). Prebuilt fixtures are excluded. Bubble Sort and BST sizes above 1,000 produce eight guarded skip rows with blank metrics, not zero-valued measurements. The controlled supplement uses unique keys range(n), capacities 1 and 2n, three sizes, and five repeats per metric: all keys share one chain in the first profile, while the second places these keys in separate buckets. Its CSV is kept separate to preserve the original workload definition.

Ordering and Rank-Selection Results

Figures 1 and 2 show the measured timing profiles for complete ordering and middle-rank selection, respectively.

At n=1,000, the observed Bubble Sort means were 26.5049 ms for random input, 0.0789 ms for sorted input, 31.4384 ms for reverse input, and 24.3214 ms for the duplicate pattern. The corresponding random-input median was 25.9588 ms, with an observed range of 25.2322–29.9177 ms. Sorted input required exactly 999 comparisons and zero swaps. The reverse sample required 499,500 comparisons and 499,476 swaps; equal repeated values explain why swaps need not equal the maximum distinct-key inversion count. These counters provide execution evidence alongside elapsed time.

The order effect is consistent with the early-exit decision: a sorted sample completes one scan, while a difficult order drives many passes. This result supports using Bubble Sort to

demonstrate local comparisons and invariants on small inputs. It does not recommend the algorithm for general large-array sorting. Guarded 5,000-element rows deliberately contain no observation and must not be connected to a zero point. The plotted finite samples cannot establish an asymptotic bound by themselves; the bound comes from the nested-pass implementation and is supported, rather than proved, by the measurements.

Quickselect's random-input mean at n=5,000 was 1.2626 ms, with median 1.2420 ms and range 0.7844–1.7544 ms. Its duplicate-pattern mean at that size was 0.7308 ms. Figure 2 displays all four patterns. Three-way partitioning can discard an entire equal region; however, five selected pivot seeds and one generated sample per pattern do not describe the full probability distribution of selection time. These values concern one middle rank. They cannot establish that selection replaces sorting when the application needs a complete ordering, nor that the randomized algorithm never encounters its quadratic worst case.

Tree Shape and Controlled Hash Collisions

Figure 3 shows successful search batches for the unbalanced tree. Figure 4 presents the separate controlled hash-capacity experiment.

For 1,000 input queries, BST search-all averaged 0.3972 ms on the random insertion sequence, 10.5124 ms on sorted insertion, and 11.0121 ms on reverse insertion. The sorted/random mean ratio was 26.47×. Because the fixture is built before measurement, the difference describes search paths in the resulting tree; it does not measure the construction cost. A monotone insertion sequence creates long paths in this unbalanced implementation. An iterative implementation remains vulnerable to that shape even though it avoids recursive stack overflow.

The duplicate-pattern fixture contains at most seven nodes regardless of the number of submitted values. Its repeated successful queries therefore concern a different effective tree size. Comparing only the input-size column would hide this fact; fixture_items in the raw data records the distinction. Input order, distinct-key count, and balance are separate variables. A stronger experiment would independently vary them, measure heights, and include unsuccessful searches. The present study makes a narrower claim about the recorded insertion sequences and successful query batches.

The controlled hash supplement at n=1,000 averaged 12.6926 ms for one bucket and 0.1861 ms for 2,000 buckets, a mean ratio of 68.20×. Figure 4 visualizes the three measured sizes. Both profiles find the same unique integer keys and return the same values. One bucket forces a linear chain search for each query; distributing these particular nonnegative integer keys across 2n buckets removes that chain scan. Construction is excluded in both cases. This demonstrates an observed collision cost without misrepresenting the default profile, which uses random/repeated values and capacity max(8,2n). It also does not measure resizing, because the custom table has fixed capacity. The larger bucket array consumes fixture storage that this incremental allocation metric excludes.

Allocation Evidence and Interpretation

Figure 5 shows incremental operation-allocation peaks for four random-input workloads.

For random input at n=1,000, mean incremental peaks were 16,232 bytes for untraced Bubble Sort, 8,056 bytes for Quickselect, 8,800 bytes for draining the Queue, and 8,872 bytes for the Hash Table find-all batch. These values describe allocations during each measured operation. The Queue and Hash Table already exist when tracing starts, so their readings cannot rank the

total memory efficiency of the two representations. Their returned lists retain n results, making output construction a substantial part of the observed peak.

The algorithm contracts also allocate protective list copies. Bubble Sort returns an immutable output tuple and counters; Quickselect returns one value after working on its copy. The public nonmutation guarantee therefore introduces O(n) storage even though a destructive selection routine could partition in place. This trade-off benefits an interactive application in which users expect the original dataset to survive a run. The report evaluates the implemented contract rather than borrowing an in-place textbook space bound that would exclude the copy.

Tracing is a separate workload and is intentionally absent from these figures. Immutable snapshots prevent later swaps from rewriting earlier visual frames, but storing an entire list for every comparison has the cubic worst-case implication discussed earlier. Reducing the trace to deltas could save storage; reconstruction would then require replay logic and a more complex step-retrieval contract. The current small-input boundary makes complete snapshots a defensible teaching choice while preserving an inexpensive untraced path for benchmarks.

Memory variability is reported through the same mean, median, minimum, and maximum summaries as timing. Repeated equal peaks can occur for deterministic allocation paths and do not imply that whole-process memory is constant. The active interpreter, object layout, allocator reuse, result lifetime, and tracemalloc's scope influence these measurements. No RSS instrument, energy measurement, or simultaneous browser-memory measurement was collected. The charts consequently label Python-tracked bytes explicitly and avoid adding native memory estimates that were never observed.

Evaluation, Limitations, and Conclusion

The portfolio's strongest design result is the separation between computational behavior and explanation. Immutable sorting records let the user move backward and forward without mutating the data. Explicit manager routes expose a consistent surface while preserving each structure's native semantics. Strict integer parsing and bounded interface operations make errors understandable. The same core functions support unit tests, interface integration tests, and command line measurement, so performance collection does not require rendering a web page. This architecture makes the results easier to inspect and reproduce.

Several trade-offs remain deliberate. Bubble Sort offers transparent local events at quadratic untraced cost; randomized Quickselect reduces expected selection work while retaining an adverse pivot sequence. The unbalanced BST preserves a simple relationship between insertion order and shape but cannot promise logarithmic search. Fixed-capacity separate chaining exposes collision behavior but requires callers to consider capacity. Dictionary-backed set membership simplifies reliable algebra, while the deque-backed Queue avoids repeated front shifts. Graph traversal supports undirected reachability rather than weighted shortest paths or directed algorithms. These choices match the educational scope without claiming a general-purpose production library.

The experiment has finite external validity. It uses one Windows machine, one Python version, five repetitions, one generated sample per size/pattern, successful structure lookups, and path-only BFS. It excludes fixture construction and caps quadratic workloads. Minimum–maximum whiskers show observed spread, not confidence intervals. Pseudorandom seeds make the dataset and pivot schedule reproducible; they do not remove scheduler noise or constitute an

adversarial search. The report therefore does not fit universal constants, claim statistical significance, or extrapolate measured values to untested hardware and sizes.

Three improvements follow directly from the evidence. A balanced tree would control insertion-order sensitivity; a resizing hash policy would manage load as the structure grows; and additional independent dataset seeds with construction, unsuccessful-query, and broader graph workloads would strengthen empirical coverage. Delta-based visualization remains a separate possible optimization with additional reconstruction complexity. The current release completes the intended portfolio and supplies a traceable path from source to raw observations, summaries, figures, report, slides, and verification records. Its findings illustrate how representation and API decisions affect the specific work measured, while maintaining a clear boundary between theory, observation, and future work.

References

OpenDSA Project Contributors. (n.d.). *Bubble sort*. In CS3 data structures & algorithms. https://opendsa-server.cs.vt.edu/ODSA/Books/CS3/html/BubbleSort.html

Python Software Foundation. (n.d.-a). *collections—Container datatypes*. Python 3.14 documentation. https://docs.python.org/3.14/library/collections.html

Python Software Foundation. (n.d.-b). *time—Time access and conversions*. Python 3.14 documentation. https://docs.python.org/3.14/library/time.html

Python Software Foundation. (n.d.-c). *tracemalloc—Trace memory allocations*. Python 3.14 documentation. https://docs.python.org/3.14/library/tracemalloc.html

Sedgewick, R., & Wayne, K. (n.d.-a). *BreadthFirstPaths.java* [Source code]. Algorithms, 4th edition companion repository. https://github.com/kevin-wayne/algs4/blob/master/src/main/java/edu/princeton/cs/algs4/BreadthFirstPaths.java

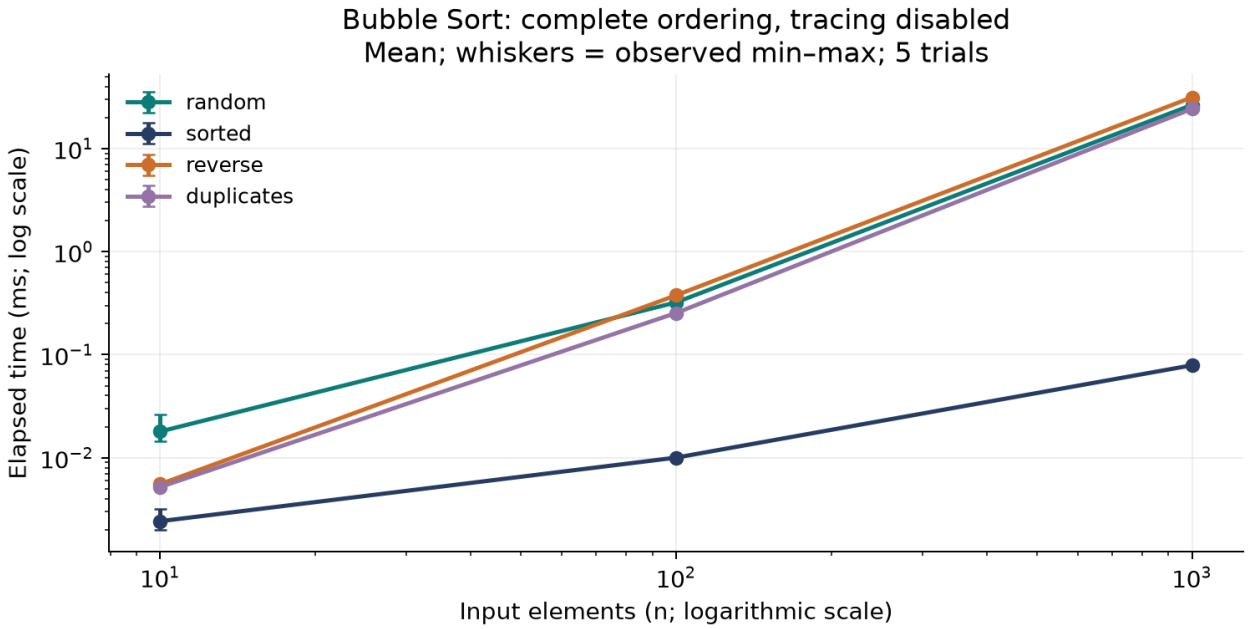
Sedgewick, R., & Wayne, K. (n.d.-b). *BST.java* [Source code]. Algorithms, 4th edition companion repository. https://github.com/kevin-wayne/algs4/blob/master/src/main/java/edu/princeton/cs/algs4/BST.java

Sedgewick, R., & Wayne, K. (n.d.-c). *Quick.java* [Source code]. Algorithms, 4th edition companion repository. https://github.com/kevin-wayne/algs4/blob/master/src/main/java/edu/princeton/cs/algs4/Quick.java

Sedgewick, R., & Wayne, K. (n.d.-d). *SeparateChainingHashST.java* [Source code]. Algorithms, 4th edition companion repository. https://github.com/kevin-wayne/algs4/blob/master/src/main/java/edu/princeton/cs/algs4/SeparateChainingHashST.java

Figure 1

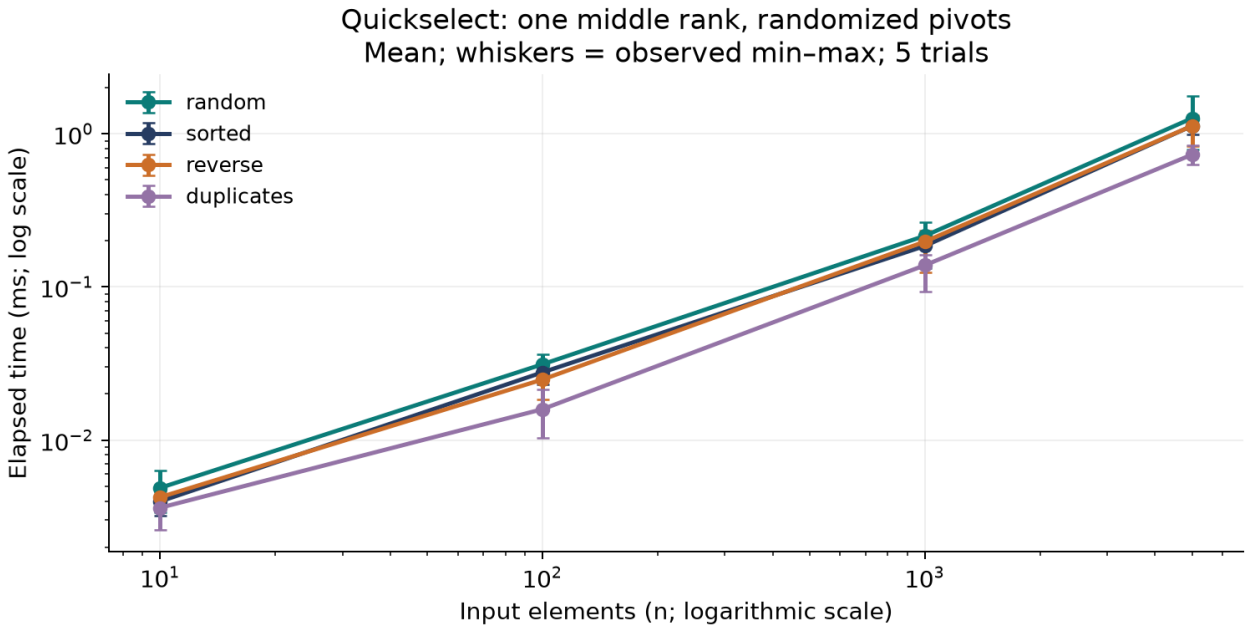
Bubble Sort Timing by Input Pattern



Note. Complete ordering with tracing disabled; cases above 1,000 elements are guarded skips. Lines show observed means; whiskers show observed minimum–maximum across five trials. Both axes use logarithmic scaling. Original figures are generated from the recorded CSV observations.

Figure 2

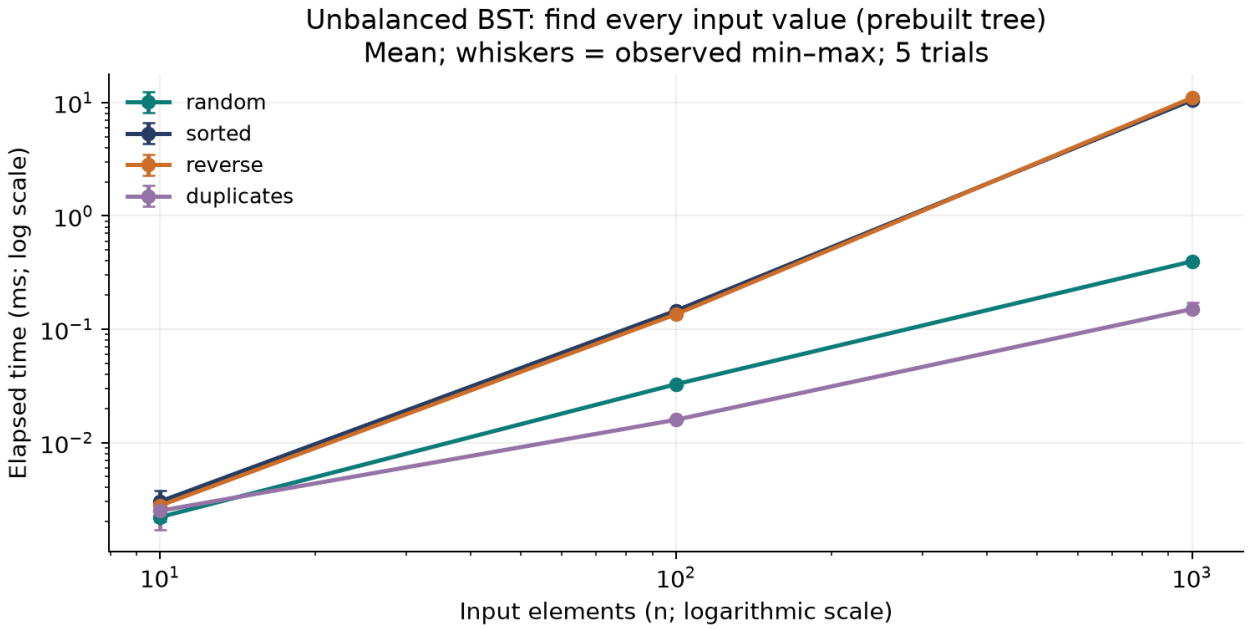
Quickselect Middle-Rank Timing by Input Pattern



Note. One-based rank k = (n + 1) // 2; duplicates occupy repeated ranks. Selection and sorting perform different tasks. Lines show observed means; whiskers show observed minimum–maximum across five trials. Both axes use logarithmic scaling. Original figures are generated from the recorded CSV observations.

Figure 3

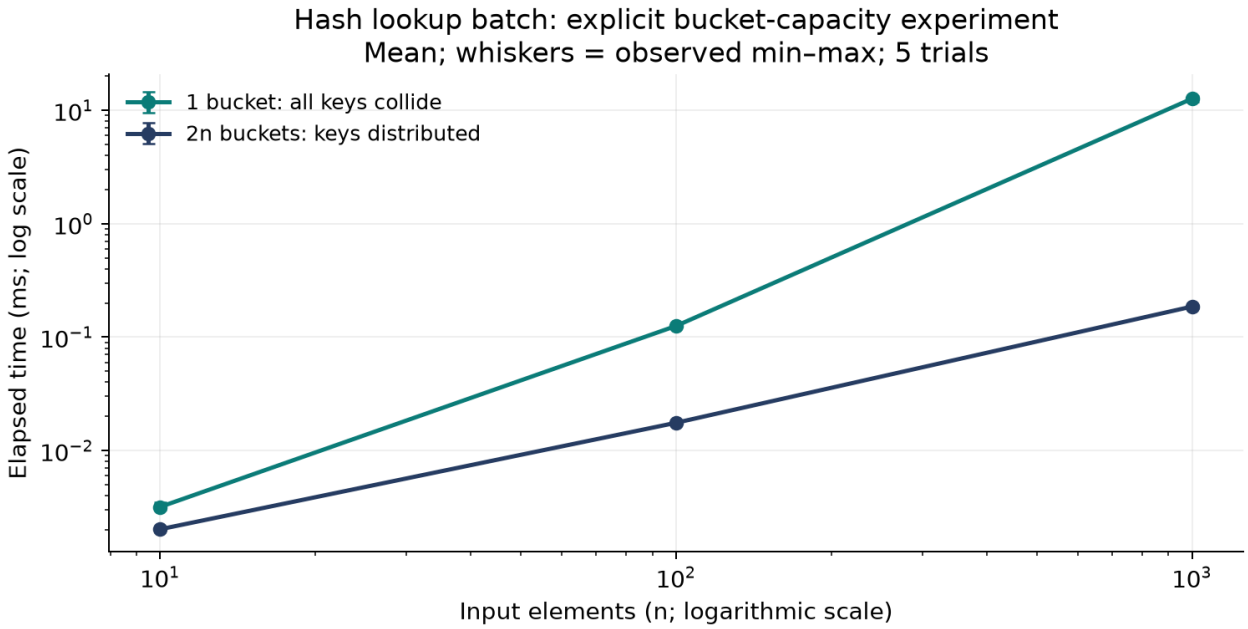
Unbalanced Binary Search Tree Search Timing



Note. Every submitted input is searched in a prebuilt unique-integer tree. Repeated keys can reduce the stored fixture size. Lines show observed means; whiskers show observed minimum–maximum across five trials. Both axes use logarithmic scaling. Original figures are generated from the recorded CSV observations.

Figure 4

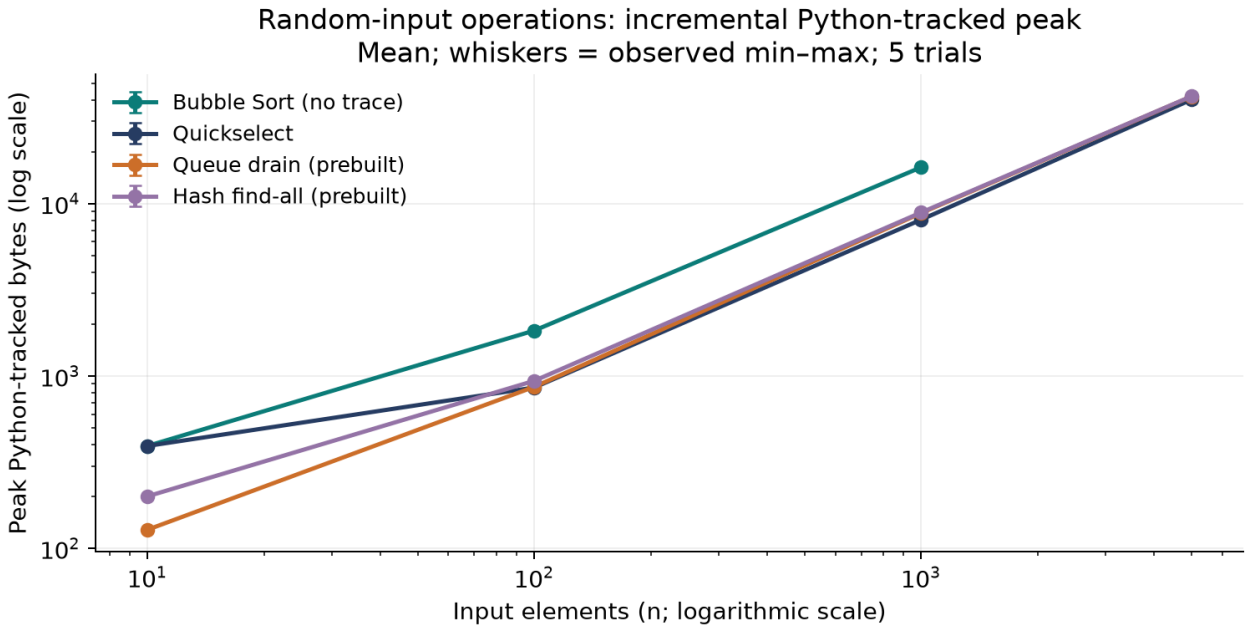
Controlled Hash Lookup Timing by Bucket Capacity



Note. A separate profile uses keys range(n), capacity 1 versus 2n, and the same successful lookup queries in both profiles. Lines show observed means; whiskers show observed minimum–maximum across five trials. Both axes use logarithmic scaling. Original figures are generated from the recorded CSV observations.

Figure 5

Incremental Python-Tracked Peak Allocation



Note. Random-input operation peaks include returned results and internal copies, exclude prebuilt fixtures, and do not measure resident RAM. Lines show observed means; whiskers show observed minimum–maximum across five trials. Both axes use logarithmic scaling. Original figures are generated from the recorded CSV observations.